Bootcamp III - Entrega 2

Publicação do projeto, refinamento e análise crítica de IA

Etapa intermediária - TaskTracker

ADS - Taguatinga / Noturno | Snapshot de 23/10/2026.

Integrante	RA
Jonathan Rodrigues Silva Corrêa	22450356
Matheus Couto Nogueira	22505474
Esther Diniz Bastos	22505492
Cauã Gonçalves Xavier Mrad	22452326
Rillary Lorranne de Souza Portilho	22450936
Gustavo Araújo do Carmo	22304113

Líder para submissão: Gustavo Araújo do Carmo - RA 22304113.

Repositório público: github.com/gustavoaraujocd/tasktrackerr.

Versão desta etapa: feature/bootcamp3-entrega2.

Revisão: PR #16, de feature para develop.

Estado de homologação: código e testes publicados para revisão. Após leitura da apostila, permanecem pendentes o experimento comparativo com dados próprios, a captura autêntica do harness e o fechamento da governança, incluindo revisão independente e promoção para main. Este PDF é um snapshot de preparação, não uma declaração de entrega concluída.

Esta entrega mantém a identidade do projeto utilizado nas etapas anteriores, mas consolida os requisitos específicos do Bootcamp III: aplicação integrada, regressões, governança, defesa técnica e análise crítica do uso de IA. Cada integrante deverá enviar o mesmo PDF consolidado conforme o enunciado.

Conteúdo do relatório

1. Solução integrada e arquitetura.
2. Execução do harness, resultados e evidências do CI.
3. Erros lógicos e refinamento da especificação.
4. Governança, sprints e relato de experiência.
5. Decisões de arquitetura e trade-offs.
6. Comparação de ferramentas, ética e segurança da IA.
7. Conferência com a apostila e estado da homologação.
8. Auditoria de requisitos detalhados do guia.

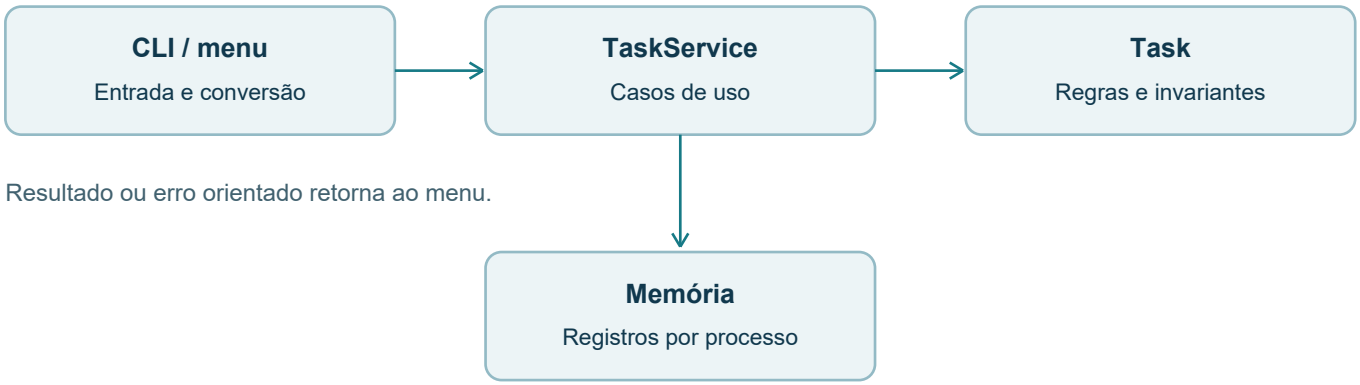
1. Solução integrada e arquitetura

O TaskTracker organiza tarefas locais com título, descrição, prioridade, prazo e status. A Entrega 1 fornecida documentou o núcleo Python e 8 testes. Na base inspecionada desta etapa havia 13 testes e um menu limitado a cadastro e listagem. A E2 conecta ao menu os demais casos de uso já presentes no serviço.

Menu	Ação
1 / 2 / 3	Cadastrar / listar / sair (opções preservadas)
4 / 5 / 6 / 7	Consultar por ID / atualizar / concluir / filtrar

Título obrigatório, prioridade e status controlados, IDs preservados e erros orientados são invariantes do domínio. Na atualização, Enter mantém o valor e '-' limpa descrição, data ou hora. O usuário copia o ID completo para consultar, editar ou concluir. Alertas aparecem no terminal para tarefas com prazo vencido e ainda não concluídas.

Fluxo implementado



Opção 3 / EOF / Ctrl+C encerram o processo e descartam os registros.

Ambiente padronizado e limites

Python 3.11+ com instalação por ambiente virtual ou Docker/Compose. A imagem runtime inicia a CLI, mantém entrada interativa e executa com UID 10001; testes têm estágio separado. O README contém o passo a passo para clonar a branch desta etapa e executar localmente ou em contêiner.

O armazenamento permanece em memória, conforme a Entrega 1. Ao encerrar, os dados são perdidos. Não há banco, login, API, múltiplos usuários ou prontidão para produção. Prazos usam calendário local sem fuso, enquanto created_at registra UTC. A decisão e seus limites estão nos ADRs.

O Codespaces é opcional e seu rebuild completo não foi homologado nesta etapa. Docker foi verificado no CI; esta distinção evita afirmar validação de um ambiente não concluído.

2. Execução e resultados dos testes

Snapshot: 23/10/2026. Base: c946e57. Revisão técnica: 984ed48daad1fd83ca9c0b7727b67e843adf5301.

Resultado consolidado

Grupo	Casos	Resultado
Domínio inicial - test_service.py	11	Aprovados
Regressões E2 - test_refinements.py	15	Aprovados
CLI em subprocesso - test_cli.py	3	Aprovados
Menu integrado - test_cli_workflows.py	10	Aprovados
Total coletado	39	39 aprovados; 0 falhas; 0 erros; 0 ignorados

Passagem: 39/39 = 100%. Não se mediu cobertura de linhas ou ramos. Passagem integral não significa ausência de defeitos, auditoria de segurança ou homologação de produto.

Ambiente e comandos

Execução local: Windows, Python 3.12.14, pytest 9.1.1. A saída final registrou 39 passed in 1.84s; duração é informativa e varia entre ambientes.

```
python -m pip install -e ".[dev]"
python -m pytest -q --junitxml=test-results.xml
docker compose build tests
docker compose run --rm tests
```

O Docker foi validado no GitHub Actions em Linux, com job Python 3.11 e job Docker separados. Não foi executado Docker neste Windows.

Ciclo antes e depois

A suíte de novos contratos, aplicada à base anterior, registrou 8 failed, 5 passed in 1.36s. Após as correções e a ampliação da integração, a suíte completa registrou 39 passed. Não são os mesmos denominadores: o primeiro resultado corresponde a 13 casos de diagnóstico; o segundo inclui os 39 casos finais.

Os resultados locais estão em docs/evidencias/antes.log, depois.log e depois.xml. O hostname foi removido da cópia pública do JUnit; resultados e casos foram preservados.

Evidência automatizada

GitHub Actions run #27 (39957156921), head 984ed48: jobs test e docker concluídos com sucesso. O checkout executado foi o merge ref 832c6ecf9f0b546984b58039311707f292ba206d, combinando esta revisão com develop. Python 3.11.17: 39 passed in 0.20s. Contêiner: 39 passed in 0.50s. O smoke test também verificou UID 10001 e o menu. Trechos reais estão em docs/evidencias/ci-resumo.log.

Execução do CI. O artefato pytest-results contém o JUnit do job Python. O job Docker valida Compose, build, UID 10001, menu na imagem runtime e suíte em contêiner.

Limites: testes não cobrem concorrência, durabilidade, integração multiusuário ou calendários entre fusos, capacidades que não fazem parte da solução local especificada.

3. Erros lógicos e loop de re-especificação

Base inspecionada: c946e5781f935f87b51b54c4f0ce56503dae69b3. Issue #14; PR #16. A suíte de contratos adicionada foi executada antes da correção: 8 falhas e 5 aprovações. Não se alega que a suíte antiga falhava: os casos de borda ainda não eram exercitados.

Evidência / causa	Antes	Contrato e correção	Teste de regressão
datetime herda date	Validação aceitava valor temporal com hora em campo de data	RN11: rejeitar datetime explicitamente	test_datetime_is_not_a_calendar_due_date
Horário isolado	Era aceito e ignorado pelo cálculo de atraso	RN12: horário exige data	test_time_without_date_is_rejected
Horário com timezone	Era aceito sem política consistente	RN13: prazo civil local sem tzinfo	test_aware_deadline_time_is_rejected
Referência aware	Comparação com deadline naive lançava TypeError	RN13: erro explícito ValueError	test_aware_reference_has_explicit_validation
Filtro inválido	Retornava lista vazia silenciosamente	RN15: None ou enum Status	test_invalid_filter_is_not_silently_an_empty_result
Texto None: dois casos	AttributeError em strip de título/descrição	RN10: checar tipo antes de normalizar	test_invalid_text_type_uses_domain_error
Remoção só da data	Update deixava horário sem efeito	RN12/RN16: validar antes de substituir	test_invalid_update_does_not_change_stored_task

Integração por inspeção

O serviço já implementava os casos RF02/RF04/RF05/RF06; o menu não oferecia acesso. As opções 4-7 corrigem essa lacuna e seus testes verificam o resultado no serviço real. A opção 3 continua encerrando, evitando quebrar fluxos anteriores.

Re-especificação

1. Falha reproduzida com entrada controlada.
2. Ambiguidade do contrato identificada (data, horário, fuso, filtro ou erro).
3. RN10-RN16 documentadas antes da correção.
4. Validação do domínio ajustada; menu integrado; regressões mantidas.
5. Harness ampliado de 13 casos da base para 39 na E2, com nenhuma falha na execução final.

Os 8 casos falhos representam sete grupos de causa, pois o caso de texto é parametrizado em título e descrição. Evidências: antes.log, depois.log e depois.xml.

Limites conhecidos

Objetos do domínio são mutáveis; uso externo pode contornar o serviço. Não há durabilidade, autenticação, concorrência ou calendário entre fusos. A E2 mantém o escopo local da Entrega 1, sem prometer capacidades ausentes.

4. Governança, sprints e comunicação técnica

Histórico comprovado

- Sprint 1: issues #1, #2 e #3, fechadas, documentam especificação, harness e ambiente.

- PR #4: aprovado por estherbastos em 12/09/2026 22:26 UTC e JON4TH4NRSC às 23:09 UTC; integrado em develop às 23:51 UTC. A aprovação de cauamrad em 13/09 16:10 UTC ocorreu depois da integração e não é apresentada como aprovação prévia.

- PR #5: encerrado sem merge por conflito histórico.

- PR #6: integração recomposta a partir da main; harness inicial de 8 testes.

- PR #8: runtime interativo, testes CLI e Docker; 13 casos.

- PR #10: ajuste de imagem de desenvolvimento por falha de chave GPG; não se afirma homologação final do Codespaces.

- Sprint 2: issues #14 e #15, abertas para aceite de código, documentação e análise ética.

- PR #16: feature/bootcamp3-entrega2 para develop; testes de regressão, integração e relatório desta etapa.

As fases acadêmicas anteriores compartilham o mesmo repositório. Os nomes históricos de PRs e commits são preservados; eles não alteram a identificação desta Entrega 2 do Bootcamp III.

Estado e critérios de aceite

Na inspeção de 23/10/2026, main e develop estavam sem proteção automática. Não é correto afirmar que o GitHub impede tecnicamente merge sem revisão. A equipe deve configurar proteção se quiser transformar a política em bloqueio obrigatório; esta entrega não altera permissões administrativas.

O PR #16 deve receber revisão real de um integrante diferente do autor. O revisor verifica: RF01-RF08 no menu, contratos RN10-RN16, atualização atômica, 39 testes e CI no SHA atual, comandos Docker, fontes do relatório e ausência de dados indevidos. Comentários devem ser resolvidos antes do aceite. Alterações relevantes posteriores exigem nova revisão. A promoção para main ocorre via outro PR após a integração em develop.

As aprovações do PR #4 comprovam colaboração histórica, mas não são reutilizadas como homologação do PR #16. Os checklists e issues da Sprint 2 só devem ser encerrados após o aceite real, sem simular participação ou retroagir datas.

Relato de experiência técnica

Síntese baseada no histórico inspecionado e nas execuções desta preparação; não é entrevista nem depoimento atribuído a cada integrante.

O principal aprendizado foi separar um núcleo testável de uma aplicação acessível: a existência de métodos no serviço não garantia que o usuário pudesse executar todos os casos pelo terminal. A integração do menu tornou esse descompasso visível e verificável.

Outro desafio foi especificar o significado de prazo. A tipagem aparente de Python não impediu datetime em campo date; timestamps com fuso e horários sem data também exigiram uma decisão explícita. Testar casos de borda antes de corrigir produziu evidências mais fortes do que apenas mostrar uma suíte verde.

No versionamento, o conflito histórico entre branches mostrou a importância de preservar conteúdo revisado e registrar como a integração foi recomposta. No ambiente, uma chave de pacote ausente mostrou que empacotar o código não elimina falhas externas. A governança precisa distinguir código testado, infraestrutura preparada e ambiente efetivamente homologado.

O apoio da IA ampliou a capacidade de produzir código, testes e documentação, mas exigiu leitura crítica. A equipe continua responsável por entender os contratos, revisar o que publica e comprovar participação por registros reais. Não há medição que permita afirmar ganho percentual de produtividade.

Próxima ação humana concreta

Revisar o diff do PR #16, executar os comandos do README e registrar aprovação ou solicitações de ajuste no GitHub com a conta de um colega. Depois, integrar por PR em develop e main e atualizar o estado desta documentação. Este registro descreve o snapshot atual e não antecipa aprovações futuras.

Complemento após recebimento do guia

O aceite não depende apenas do review. A apostila detalha experimento empírico de IA, marco da sprint, participação dos integrantes, evidências visuais e conferências. Consulte auditoria-guia-e2.md. Não há afirmação de que essas atividades já ocorreram. Líder para submissão: Gustavo Araújo do Carmo.

5. Defesa das decisões de arquitetura

Consolidação para Bootcamp III / Entrega 2, 23/10/2026. As decisões da base são preservadas; os refinamentos E2 seguem para homologação no PR #16.

ADR-001 - Python 3.11+

Contexto: equipe necessita CLI simples, biblioteca de datas e testes automatizados. **Decisão:** Python 3.11+ com biblioteca padrão no runtime. **Alternativa:** framework web e front-end separado. **Consequências:** menor número de dependências e execução simples; ausência de interface gráfica e distribuição depende de Python ou Docker.

ADR-002 - Domínio separado da CLI

Contexto: regras precisam ser testadas sem interação manual. **Decisão:** entidades e enums em models.py; casos de uso e memória em service.py; I/O em main.py. **Alternativa:** um script único contendo interface e regras. **Consequências:** testes do domínio independentes de input; é necessário testar também integração do menu. O serviço não impõe imutabilidade aos objetos retornados.

ADR-003 - Memória e escopo local

Contexto: Entrega 1 especificou foco em núcleo funcional e harness, sem persistência. **Decisão:** manter dicionário por processo e explicitar perda dos registros ao encerrar. **Alternativa:** SQLite ou arquivo JSON. **Consequências:** não exige instalação de banco; sem durabilidade, múltiplos usuários ou controle de concorrência. Um volume Docker isolado não torna a aplicação persistente. Persistência exigiria nova especificação e testes.

ADR-004 - Pytest e regressões

Contexto: inspeção sozinha não provou todos os contratos. **Decisão:** manter suíte inicial e incluir casos reproduzíveis de borda, subprocesso e menu com serviço real. **Alternativa:** só execução manual ou testes gerados sem revisão de assertivas. **Consequências:** 39 casos aprovados no snapshot; os resultados são evidência limitada aos cenários exercitados. Há risco de testes confirmarem os mesmos pressupostos errados do código; o revisor deve conferir regra e oráculo.

ADR-005 - Docker em estágios

Contexto: imagem inicial executava apenas testes e faltava interação com CLI. **Decisão:** runtime separado de tests, stdin/TTY no Compose e usuário UID 10001 no runtime E2. **Alternativa:** instalar ferramentas de desenvolvimento na imagem de produção e executar como root. **Consequências:** menor superfície e privilégio no runtime; build de testes depende de rede. Base e dependências sem pinagem integral por hash ainda permitem variação. Docker não é prova de segurança ou isolamento absoluto.

ADR-006 - Feature / develop / main

Contexto: é preciso revisar antes de integrar. **Decisão:** PR com problema, requisitos, resultados e aceite; aprovação humana de colega diferente do autor e CI no SHA atual. **Alternativa:** push direto na main. **Consequências:** rastreabilidade e revisão independente; integração depende de disponibilidade da equipe. Sem branch protection, a política pode ser contornada e exige disciplina. Aprovação antiga não homologa código posterior.

ADR-007 - Prazos civis locais e timestamps UTC

Contexto: datetime herda date; horário sem data era ignorado; referências aware causavam TypeError. **Decisão E2:** rejeitar datetime como due_date, exigir data para hora e rejeitar tzinfo nos prazos e referências; created_at segue UTC-aware. Sem hora, usar fim do dia, com comparação estritamente maior. **Alternativa:** converter todos os prazos para UTC com timezone obrigatório. **Consequências:** comportamento explícito para CLI local e erro de domínio orientado; não atende agendas entre fusos ou regras de horário de verão. Esses casos exigem novo contrato, não conversão silenciosa.

ADR-008 - IA assistiva e homologação humana

Contexto: geração rápida pode reproduzir falhas e produzir explicações sem evidência. **Decisão E2:** SDD orienta mudanças; IA ajuda a implementar e testar; colega revisa o diff, o significado dos testes e as fontes antes do merge. **Alternativa:** aceitar automaticamente a saída porque o harness está verde. **Consequências:** mais esforço de leitura e investigação, com responsabilidade humana explícita. AGENTS.md orienta o agente, mas não substitui sandbox, controle de acesso ou segurança do sistema operacional.

6. Análise comparativa e ético-técnica

Bootcamp III - Entrega 2 | TaskTracker | 23/10/2026.

Estado após leitura da apostila

A apostila (p.17-18) exige comparação empírica. Esta seção documental é apoio e não atende sozinha a esse item. O experimento ainda não foi executado; o protocolo está em experimento-ia.md. Não há resultados de duas ferramentas ou configurações para declarar.

Método e limites da comparação

Uso comprovado neste projeto: Codex, com apoio do ChatGPT na organização técnica. O responsável confirmou que não utilizou Claude Code, Cursor ou Antigravity. A matriz abaixo é uma avaliação documental de capacidades e adequação ao fluxo; não é experimento controlado, ranking de qualidade ou medição de produtividade. Não foram medidos tempo economizado, custo, taxa de acerto ou comparação entre modelos. A interface Codex utilizada no trabalho é a aplicação integrada; não se afirma que todos os comandos foram executados especificamente pelo produto Codex CLI.

As fontes dos fabricantes foram consultadas em 23/10/2026. Suas afirmações descrevem recursos, não uma auditoria independente de segurança. A configuração efetiva varia por versão, sistema operacional, plano e política da organização.

Matriz comparativa

Ferramenta / evidência	Pontos fortes relevantes	Limitações e riscos	Impacto no TaskTracker
Codex / Codex CLI - uso real do ecossistema Codex; CLI analisada documentalmente	Pode editar arquivos e executar testes; separa sandbox de política de aprovação. Adequado para ciclos especificar, alterar, testar e inspecionar diff. [S1]	A geração pode manter premissas erradas. Permissões amplas, rede e conectores precisam de controle. Harness verde não significa contrato completo.	Apoio verificável à CLI, harness, Docker, README e refinamentos. A suíte cresceu de 8 casos registrados na Entrega 1 para 39 no snapshot E2; esse aumento é evidência de cobertura de cenários, não prova causal de superioridade da IA.
Claude Code - apenas documentação	Arquitetura de permissões e sandbox para comandos; políticas permitem ajustar o nível de autonomia. [S2]	Aprovação de comando e isolamento do SO são camadas diferentes. Saídas continuam exigindo revisão; não há qualidade local medida.	Poderia executar o mesmo fluxo SDD e testar mudanças. O impacto real neste código não foi avaliado.
Cursor - apenas documentação	Ferramentas de leitura, edição e terminal integradas; revisão pode ser associada ao contexto do código. [S3]	A documentação alerta que alterações podem executar por auto-reload e que guardrails são melhores esforços. Edição salva em disco exige versionamento e revisão.	Potencial para revisão contextual das regras; nenhum resultado experimental ou ganho de qualidade é atribuído ao projeto.
Antigravity - apenas documentação	Permissões Deny/Ask/Allow e mecanismos para revisar mudanças e comentar diffs. [S4, S5]	A autonomia e o isolamento dependem do preset. Permissões amplas aumentam exposição; capacidade de apresentar artefatos não prova correção.	Poderia tornar planos e alterações inspecionáveis. Nenhuma execução comparativa foi feita neste repositório.

A decisão de continuar com Codex se apoia no histórico e contexto já disponíveis. Para comparar desempenho no futuro: usar a mesma especificação e base imutável, os mesmos casos de teste independentes, registrar versões, tempo humano de revisão, falhas introduzidas e custo. Variar apenas a ferramenta, repetir execuções e publicar resultados negativos também.

Alucinação e confiança indevida

Uma IA pode inventar métodos de bibliotecas, formatos de API, resultados de testes, aprovações ou requisitos. Código sintaticamente válido também pode ser logicamente errado. Neste projeto, a aceitação de datetime como date e o horário sem data demonstraram como uma implementação plausível pode contrariar o contrato. Não há evidência suficiente para atribuir a autoria de cada defeito exclusivamente à IA.

O controle adotado é transformar a alegação em cenário verificável: entrada, saída ou erro esperado e invariante. O teste deve expressar a regra de negócio e falhar na implementação defeituosa. Na E2, os novos contratos produziram 8 falhas antes dos ajustes. Preservar a suíte original evita substituir testes incômodos por outros que só confirmem o código novo. O revisor humano precisa conferir o oráculo, porque agente e testes podem compartilhar o mesmo erro.

Código inseguro, comandos destrutivos e prompt injection

O risco do agente excede o texto que ele gera: comandos podem apagar arquivos, expor tokens, alterar permissões ou instalar dependências maliciosas. Um README ou log de terceiro pode conter instruções maliciosas para executar comandos ou enviar arquivos. Esse conteúdo é dado de análise, não autorização para mudar o objetivo da tarefa.

Para o TaskTracker: trabalhar em branch, inspecionar diffs, limitar escrita ao workspace, reduzir rede e evitar credenciais de produção. Comandos irreversíveis precisam de revisão explícita do alvo e do efeito. Não desabilitar verificações de assinatura para contornar falhas de instalação. O episódio documentado de chave GPG ausente no Codespaces foi tratado pela troca da imagem de desenvolvimento, sem desativar a validação; o rebuild completo não foi homologado após o cancelamento do ambiente.

O runtime E2 usa UID 10001; o CI tem contents:read e tempo limite. Esses controles reduzem privilégio e exposição, mas não tornam Docker uma fronteira absoluta. Dependências e imagens sem pinagem integral continuam sendo risco de cadeia de suprimentos. Não se executou pentest ou auditoria de dependências nesta entrega.

Privacidade e confidencialidade

Prompts, arquivos selecionados, diffs e logs podem conter dados pessoais, segredos ou propriedade de terceiros. Rodar comandos localmente não implica que o contexto enviado a um serviço de IA permaneça local. Políticas de retenção, treinamento e subprocessadores precisam ser verificadas no contrato e nas configurações vigentes; não se presume sigilo só porque o repositório é privado.

Como prática do projeto: utilizar tarefas fictícias, selecionar apenas os arquivos necessários, remover tokens de logs e não inserir .env, chaves privadas, credenciais ou dados de clientes em prompts e commits. Nomes e RAs são dados de identificação exigidos pela atividade; o relatório não adiciona contatos, documentos ou credenciais. Os textos livres de tarefas podem conter dados sensíveis: isso deve ser evitado mesmo com armazenamento apenas em memória.

Se houver exposição, interromper o compartilhamento, revogar o segredo, avaliar os acessos e tratar o incidente com o responsável; apagar um commit recente não revoga uma credencial nem elimina cópias. O princípio de necessidade e as medidas de segurança da LGPD são referências para minimizar exposição. [S6] Esta análise não certifica conformidade jurídica ou contratual.

Propriedade intelectual e direitos autorais

Código gerado não deve ser tratado como automaticamente original, livre de restrições ou de titularidade garantida. É necessário separar as contribuições humanas, saídas do modelo, bibliotecas e trechos de terceiros. Verificar semelhanças relevantes, origem, licenças e avisos; manter atribuições de dependências e decisões sobre reutilização.

O INPI referencia a Lei de Software e a Lei de Direitos Autorais para a proteção de programas no Brasil. [S7] A discussão do U.S. Copyright Office sobre autoria e IA é referência comparativa estrangeira, não regra brasileira aplicada automaticamente. [S8] Não se atribui titularidade jurídica conclusiva à ferramenta ou à equipe sem análise da origem e dos contratos.

Não há LICENSE explícita na árvore inspecionada. Um repositório público permite consulta, mas a publicidade, por si, não demonstra autorização irrestrita para redistribuir ou comercializar o código. A equipe precisa deliberar uma licença compatível com todas as contribuições; o agente não escolhe uma em nome dos demais titulares.

Homologação humana obrigatória no SDD

1. Responsável humano confirma o problema, o escopo e os contratos.
2. Agente propõe implementação vinculada à issue e à especificação.
3. Harness e inspeção verificam cenários normais, bordas e regressões.
4. Revisor independente lê o diff, executa testes e verifica segurança e fontes.
5. Equipe aprova o comportamento e os trade-offs, resolve observações e registra decisão no PR.
6. Merge em develop e depois em main exige novo aceite compatível com o SHA integrado.

AGENTS.md é orientação textual; sandbox e permissões são controles técnicos distintos. IA não pode emitir a aprovação de um colega, fabricar colaboração ou converter resultado de teste em homologação de produto. No snapshot E2 o PR #16 está aberto, sem aprovação humana independente confirmada. A condição será atendida somente com revisão real, mantendo a rastreabilidade.

Critério de responsabilidade

A gestão de riscos proposta é contínua: definir responsáveis, mapear os dados e ações expostos, medir por testes e inspeção e tratar riscos residuais. Essa organização se alinha às funções Govern, Map, Measure e Manage do NIST AI RMF. [S9] Na prática, o critério é conseguir explicar uma mudança sem depender da autoridade do modelo e reproduzir a evidência que a sustenta.

Fontes oficiais

- S1 - OpenAI: Agent approvals & security.
- S2 - Anthropic: Claude Code Security.
- S3 - Cursor: Agent Security.
- S4 - Google: Antigravity Permissions.
- S5 - Google: Review changes.
- S6 - Brasil: Lei 13.709/2018 (LGPD).
- S7 - INPI: legislação sobre programas de computador.
- S8 - U.S. Copyright Office: Copyright and Artificial Intelligence.
- S9 - NIST: AI Risk Management Framework.

Consulta documental: 23/10/2026. Recursos e políticas dos produtos podem mudar.

7. Conferência de submissão

Item exigido	Evidência e estado neste snapshot
Repositório público e equipe	Link ativo e seis integrantes identificados neste relatório
Solução integrada	Código publicado na branch da E2, com menu para RF01-RF08
Test harness integralmente aprovado	39/39 no snapshot técnico; logs, JUnit e CI vinculados
Refinamento e defesa	Falhas antes/depois, RN10-RN16, oito ADRs e limites explícitos
Comparação documental e ética	Material de apoio pronto; falta experimento comparativo com dados próprios
Colaboração histórica	PR #4 com duas aprovações anteriores ao merge; issues e PR #6
Homologação da nova revisão	Pendente de revisão humana real no PR #16; não simulada
Publicação da nova versão em main	Pendente de aceite e promoção via develop e novo PR
Requisitos detalhados da apostila	Auditoria a seguir: governança final, captura do harness e conferências ainda pendentes

O material precisa ser complementado com o experimento real e as evidências finais descritas na auditoria. A equipe deverá revisar o diff, registrar o aceite, concluir a governança e atualizar este snapshot antes da submissão. Aprovações históricas não homologam esta versão.

Base documental fornecida pelo estudante: Entrega_1_TaskTracker_FINAL_Moodle_Bootcamp_III.pdf, oito páginas. Evidências do GitHub e fontes oficiais consultadas em 23/10/2026. A revisão futura deve preservar a distinção entre fatos observados, capacidades documentadas e hipóteses.

8. Adequação ao guia do professor

Fontes fornecidas: Apostila_Bootcamp_III_Entrega_02.pdf (26 páginas), Tutorial_StudyRoom_Bootcamp_III.pdf (21 páginas) e studyroom-api.zip. O ZIP foi lido como exemplo, sem executar código, hooks ou reaproveitar histórico Git do StudyRoom.

O projeto permanece TaskTracker

O tutorial orienta transpor o método, não copiar o sistema StudyRoom. A completude é determinada pela SPEC do projeto. Não é necessário converter a CLI em API de reservas, adotar FastAPI ou reproduzir os 23 testes do exemplo.

Preparado no TaskTracker

- RF01-RF08 acessíveis no menu e 39 testes aprovados.
- README, diagrama, oito ADRs e análise ético-técnica.
- ERROS.md com sete entradas reais e rastreabilidade E/RN/testes/PR.
- Registro R-E2-01 a R-E2-05 na especificação.
- DEMO.md com fluxo completo e critérios de observação.
- Logs antes/depois, JUnit e extrato de CI; log integral do job Docker anexado como logs_testes_final.txt.
- Líder identificado: Gustavo Araújo do Carmo, RA 22304113.
- Protocolo de experimento preparado em experimento-ia.md.

Pendências reais segundo o guia

Item	Estado e ação necessária
Comparação com dados próprios (apostila p.17-18 e 23)	Não executada. Aplicar o protocolo; matriz documental é só apoio. Duas configurações do Codex são alternativa prevista se outra ferramenta for inviável.
Participação e revisão da equipe (p.11-12)	PR #16 sem aprovação independente confirmada. Não fabricar autores, comentários ou aprovações. Conferir a participação efetiva de todos.
Publicação estável (p.3 e 23)	Código E2 está na feature. Integrar via develop e main após aceite.
Milestone, fechamento e proteção (p.5-6 e 23)	Issues #14/#15 seguem abertas; milestone da Sprint 2 e proteção de branches não foram configurados nesta preparação. O registro atual não deve afirmar que existem.
Evidência visual do harness (p.21)	Logs reais estão disponíveis; falta captura autêntica com comando e resumo visíveis no mesmo quadro. Não produzir imagem simulada de terminal.
Versão submetida (p.21-23)	Commit técnico está identificado, mas tag/release de submissão deve ser criada após a homologação final; não marcar versão ainda incompleta como v2.0.
Conferência externa e em equipe (p.14, 16 e 22)	Falta confirmar clone limpo por pessoa externa e leitura final do PDF por outro integrante. O relato atual é síntese técnica dos registros, não depoimento coletivo.
Termos da ferramenta efetivamente usada (p.19)	Identificar o plano/contrato efetivo para conferir retenção, treinamento e titularidade aplicáveis; a documentação genérica de segurança não prova as condições da conta.

A apostila acrescenta detalhamento que não estava explícito no enunciado anterior. A entrega não tem somente a pendência de review: comparação prática, evidências e fechamento da governança também precisam ser concluídos.

Histórico e autenticidade

Os números de reservas e resultados do StudyRoom são ilustrativos ou relativos àquele exemplo. Não pertencem ao TaskTracker. Nenhuma reunião, função dos colegas, execução de outra IA ou revisão por pares deve ser inventada para preencher o checklist.